BIT1123 OBJECT ORIENTED PROGRAMMING

SELF-REFLECTIVE REPORT

Object-Oriented Programming Fundamentals in Java

Student Name	BALFAQIH ABOBAKR MOHAMMED ABOBAKR
Student ID	202505010363
Class Code	202605F0782
Programme	BCSSE
NRIC / Passport No.	14213550
Course	BIT1123 Object Oriented Programming
Assignment	Assignment 1 - Individual (20%)
Repository	github.com/abocom64-sketch/Balfaqih-Abobakr-OOP

Table of Contents

> *This report reflects on the practical work stored in one GitHub repository. The weekly folders progress from basic Java syntax to classes, inheritance, encapsulation, abstraction, file handling, and a Swing graphical interface.*

1. Introduction

Object-Oriented Programming has been one of the most important parts of my development as a computing student because it changed the way I think about programs. Instead of seeing a program only as a sequence of statements, I learned to organize data and behavior into objects that represent real entities. Throughout the tutorials, I practised Java syntax and gradually applied the main OOP principles through small, focused programs.

The work in my repository records this progression. The early tutorials introduced variables, conditions, methods, classes, and objects. The middle tutorials focused on inheritance, encapsulation, abstraction, and polymorphism. The final tutorials added collections, file input/output, exception handling, and event-driven GUI programming. Managing all work in one GitHub repository also helped me understand version control, consistent folder organization, and the importance of clear documentation.

2. Knowledge Gained from Tutorials 1-10

Week 1 - Java Fundamentals and Decision Making

I began with **HelloWorld.java**, which introduced the structure of a Java class, the **main** method, and console output using **System.out.println**. In **StudentGrade.java**, I used an integer variable and an if-else-if structure to convert marks into grades. This showed me how a program selects one path according to a condition.

> ***Personal reflection:*** *The grading exercise helped me understand that conditions must be ordered carefully. Checking the highest range first prevents a high mark from matching a lower grade range.*

Week 2 - Classes, Objects, Constructors, and Methods

I created a **Student** class containing name, age, and GPA fields. A constructor initialized the state of each object, while methods such as **displayInfo()**, **study()**, and **takeExam()** represented behavior. The **Main** class created a Student object and called its methods.

> ***Personal reflection:*** *This was the point where objects became practical to me. I understood that one class is a blueprint and that many objects can later be created with different values.*

Week 3 - Inheritance and Reuse

The **Person**, **Student**, and **Lecturer** classes introduced inheritance. Student and Lecturer extend Person, so common fields such as name and age and the basic introduction behavior do not need to be duplicated. The **super** call connects each subclass constructor to the parent constructor.

> ***Personal reflection:*** *Inheritance showed me how a good class hierarchy reduces repetition. I also learned to place only truly shared state and behavior in the parent class.*

Week 4 - Method Overriding and Specialized Behavior

The same hierarchy demonstrated method overriding. Student and Lecturer each provide their own version of **introduce()**, while the original method remains available in Person. The **@Override** annotation helps the compiler verify that the subclass method correctly matches the inherited method.

> ***Personal reflection:*** *Overriding helped me see that related objects can share a common operation but perform it in a way that fits their own responsibility.*

Week 5 - Encapsulation and Controlled Access

I made the Student fields private and accessed them through getter and setter methods. The student ID is readable through a getter but has no setter, which prevents it from being changed after object creation. The accompanying documentation explains how access modifiers protect data from uncontrolled changes.

> ***Personal reflection:*** *This tutorial made encapsulation more than a definition. I learned that a class should protect its own state and expose only the operations that other classes actually need.*

Week 6 - Extending a Base Class

The **Employee** class stores a shared employee ID and name, while **Lecturer** extends it with subject and department. The Lecturer constructor uses **super(id, name)**, and the program combines inherited **displayInfo()** behavior with the subclass method **displaySubject()**.

> ***Personal reflection:*** *This exercise strengthened my understanding of constructor chaining and the relationship between protected members, inherited methods, and subclass-specific data.*

2. Knowledge Gained from Tutorials 1-10 (continued)

Week 7 - Abstraction and Polymorphism

I created an abstract **Appliance** class with shared power controls and an abstract **operate()** method. WashingMachine, Refrigerator, AirConditioner, and Microwave each override operate with their own behavior. In Main, all four objects are stored using the Appliance reference type.

> ***Personal reflection:*** *This tutorial connected abstraction and polymorphism. The main program can treat every device as an Appliance while Java chooses the correct overridden operation at runtime.*

Week 8 - Collections and User Input

The task application uses **ArrayList<String>** to store a flexible list of tasks and **Scanner** to receive console input. A loop adds three tasks, and another loop displays them with their positions. This improved my understanding of collections, loops, and generic types.

> ***Personal reflection:*** *Using an ArrayList was more practical than separate variables because the same logic can process all tasks. It also prepared me to work with collections of custom objects later.*

Week 9 - File Handling and Exception Management

The same application saves tasks with **BufferedWriter** and loads them with **BufferedReader**. Try-with-resources closes files automatically, while catch blocks handle **IOException** and display a useful message instead of allowing the program to fail without explanation.

> ***Personal reflection:*** *File handling taught me that programs often need persistence beyond one execution. It also showed why checked exceptions must be handled thoughtfully when external resources are used.*

Week 10 - Swing GUI and Event-Driven Programming

The final project is a small Programming Quiz Battle interface. The **Questions** class encapsulates the question, two options, and correct answer. **QuizBattleGUI** extends JFrame, creates labels and buttons, and implements **ActionListener**. Button events are processed in **actionPerformed()** to provide immediate feedback.

> ***Personal reflection:*** *Building a GUI was an important step because the program no longer runs only from top to bottom. It waits for the user and responds to events, which required a different way of thinking about program flow.*

3. Challenges Encountered

One early challenge was understanding the difference between a class and an object. I sometimes focused on the values inside one example instead of designing a reusable blueprint. Constructors, the **this** keyword, and object creation syntax became clearer only after I traced how values moved from Main into the object's fields.

Inheritance introduced another challenge. I needed to understand which members belong in the parent class, how **super** initializes inherited data, and why an overridden method is selected at runtime. Access modifiers also required attention because public, private, and protected members affect how classes communicate.

File input/output was difficult because the program depends on an external file and can encounter errors that ordinary calculations do not. I had to learn try-with-resources, IOException handling, and the importance of running the program from the correct folder. In the GUI tutorial, positioning components and connecting buttons to ActionListener events were new concepts.

4. How I Overcame the Challenges

I overcame these challenges by breaking each program into small parts and testing one idea at a time. I compiled the files inside each weekly folder, read compiler messages, and corrected class names, constructor parameters, and method signatures systematically. I also used console output to confirm the state of objects before adding more behavior.

For inheritance and polymorphism, I drew simple parent-child relationships and traced which constructor and method would run. For encapsulation, I asked whether another class should be allowed to change each field directly. For file handling, I separated saving and loading into their own methods and handled errors close to the operation that could fail. In Swing, I first created the window and controls, then added the event logic after the layout was visible.

5. Improvements in Java Programming Skills

My Java skills improved from writing a single main method to coordinating several classes with distinct responsibilities. I am now more comfortable with constructors, methods, access modifiers, inheritance, abstract classes, overriding, collections, file streams, exceptions, and basic Swing components. I also became more consistent with indentation, naming, folder organization, and separating model logic from interface logic.

6. Understanding of Object-Oriented Programming Concepts

Concept	Understanding demonstrated in the tutorials
Encapsulation	Private Student fields are accessed through controlled getter and setter methods.
Inheritance	Student and Lecturer reuse Person behavior; Lecturer also extends Employee.
Abstraction	The abstract Appliance class defines common operations and requires subclasses to implement operate().
Polymorphism	Appliance references invoke the overridden operate() method of each concrete appliance.
Separation of responsibilities	The quiz separates question data and answer checking from the Swing user interface.

The main lesson I gained is that OOP principles work together. Encapsulation protects an object's state, inheritance organizes shared behavior, abstraction focuses attention on essential operations, and polymorphism lets a program work with a general type while supporting specialized behavior. These principles make programs easier to extend and maintain when they are applied with clear responsibilities.

7. Future Learning Plans

My next goal is to strengthen these foundations through larger applications. I plan to learn interfaces, packages, custom exceptions, generics, Java Collections in greater depth, unit testing with JUnit, and database access with JDBC. I also want to improve the quiz project by supporting multiple questions, scoring, reusable layouts, and persistent results. Later, I would like to study design patterns and build projects with a clearer separation between the model, user interface, and data access layers.

8. Conclusion

Completing Tutorials 1-10 gave me a practical foundation in Java and Object-Oriented Programming. The sequence from basic syntax to a graphical quiz application showed my progress clearly and helped me connect theory with working code. I learned that successful programming is not only about producing output; it also requires organizing classes, protecting data, handling errors, documenting work, and improving code through testing. The repository now serves as a record of my learning and a foundation for more advanced software development.

9. GitHub Repository URL

https://github.com/abocom64-sketch/Balfaqih-Abobakr-OOP

> *The repository contains one consistent folder structure for all tutorial work, the professional README file, and this reflective report in PDF format.*